Passwordless Login for Storefront Next

Storefront Next supports passwordless login by using the Shopper Login and API Access Service (SLAS). Shoppers can securely log in by using a one-time passcode (OTP) or a magic link delivered by email or SMS.

If enabled, passwordless login is the default login mode. Hybrid storefronts don't support passwordless login.

Contents

Prerequisites

- Configure a SLAS private client with passwordless login enabled. For instructions, see Configure a SLAS Private Client in the *B2C Commerce API Developer Guide*.
- In `config.server.ts`, set `commerce.api.privateKeyEnabled` to true. If false, passwordless login is disabled at run time, regardless of the `features.passwordlessLogin.enabled` flag.
- Shoppers must log in to your site with their username and password at least once before they can use passwordless login.

Configure Settings

To control passwordless login settings, update `features.passwordlessLogin` in `config.server.ts`.

```
passwordlessLogin: {
    enabled: false,
    mode: 'email',
```

passwordless login is enabled (`true`) or not.

- `mode`: The mode used to deliver the OTP or magic link to the shopper (`email`, `callback`, or `sms`).
- `callbackUri`: The relative or absolute URI that SLAS posts shopper credentials to. Required when `mode` is set to `callback`.
- `landingUri`: An additional path to `/login` that's embedded in the magic link that shoppers click to authenticate.

The OTP code length is a shared auth setting. Valid values are 6 or 8. The value must match the OTP length configured in your SLAS client.

```
auth: {
    otpLength: 8,
}
```

You can override configuration options by using MRT environment variables with the `PUBLIC__` prefix.

```
PUBLIC__app__features__passwo
PUBLIC__app__features__passwo
PUBLIC__app__features__passwo
PUBLIC__app__auth__otpLength=
```

Configuration Options

This table lists the environment variables that override the passwordless login defaults in `config.server.ts`. Each variable is optional–omit it to keep the default.

Variable
`PUBLIC__app__features__passwordl`
`PUBLIC__app__features__passwordl`

`PUBLIC__app__features__passwordl`
`PUBLIC__app__features__otpReques`
`PUBLIC__app__features__otpReques`
`PUBLIC__app__features__resetPass`
`PUBLIC__app__features__resetPass`
`PUBLIC__app__features__resetPass`
`PUBLIC__app__auth__otpLength`

Delivery Modes

Email Mode (SLAS Native Email Service)

When `mode` is set to `email`, SLAS sends the OTP or magic link email without third-party integration. This passwordless login method doesn't require a callback URI.

1. To use this method, configure SLAS for passwordless login with email. See Passwordless Login

```
passwordlessLogin: {
        enabled: true,
        mode: 'email',
        landingUri: '/login',
    }
```

3. Set `commerce.api.privateKeyEnabled` to `true`

Callback Mode (Custom Delivery)

When `mode` is set to `callback`, SLAS makes an HTTP POST to your callback URI. The POST request includes the shopper's email and a token. Your server must then send the token to the shopper by using email, SMS, or any other channel. You can use this mode with an external callback URL or a custom POST endpoint in your storefront.

After setting `callbackUri` in `config.server.ts`, you must also allowlist the full callback URL in the SLAS client configuration:

1. Access SLAS Admin UI: Log in to your Salesforce B2C Commerce instance and go to the SLAS Admin section.
2. Create or edit a client: Select your SLAS client where you want to manage callback URLs.
3. Add allowed URLs: In the client configuration, provide a list of the valid callback URLs for your site.

Option 1 — External Callback URL

Set `callbackUri` to any publicly accessible external endpoint that can process the token and deliver it to the shopper. Examples include a B2C Commerce instance, a Managed Runtime environment, or your own server.

1. Set the full external URL directly in `config.server.ts` or by using an MRT environment variable.

2. Register the same URL in the SLAS Admin UI Callback URL field.

Option 2 — Custom POST Endpoint in Your Storefront

Storefront Next includes a callback handler at the path configured in `callbackUri` (default: /passwordless-login-callback). Replace the logic inside `handlePasswordlessCallback` in `src/lib/passwordless-login.ts`. The function receives the validated `email_id` and token from SLAS. Build and send your own message with a service provider.

Best Practices for Passwordless Login

- Implement rate limiting to prevent abuse of the passwordless login feature. For example, if you use embedded CDN (eCDN), you can apply eCDN Rate Limiting Rules.
- Use security measures to protect shopper accounts. For example, if you use a third-party content delivery network (CDN) such as Cloudfare, you can block requests by Threat Score.
- Track shopper usage of authentication options and diagnose issues efficiently. For example, use Log Center to access logs from Managed Runtime (MRT) and your B2C Commerce instance.

Try for free

MuleSoft
Tableau
Commerce Cloud
Lightning Design System
Einstein
Quip

Component Library
APIs
Trailhead
Sample Apps
AppExchange

Events
Partne
Blog
Salesfc
Salesfc